

Performance Testing

Date	8 July 2026
Team ID	xxxxxx
Project Name	Personalized Networking Assistant
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Manual Testing, FastAPI Swagger UI, Browser Testing
Type of Testing	Performance Testing, Functional Testing
Target Module / API	FastAPI Backend APIs and Streamlit Frontend
Test Environment	Local Machine (Windows), Python Environment
Test Date	8-07-2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	User enters profile details and generates networking message	1	3 sec	AI-generated message should be displayed successfully
2	Multiple API requests sent to generate recommendations	5	10 sec	System should handle requests without failure
3	Fact checking request using Wikipedia API	1	5 sec	Relevant fact-check result should be returned

4	History retrieval from JSON storage	5	5 sec	Previous conversations should load correctly
---	-------------------------------------	---	-------	--



Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 5 seconds	~3 seconds	Pass	API responses generated successfully
2	Response Time (Max)	< 10 seconds	~7 seconds	Pass	Higher response time observed during AI generation
3	Throughput (Req/sec)	>1 req/sec	~2 req/sec	Pass	Application handled requests successfully 4 Er
4	Error Rate	< 1%	0%	Pass	No critical errors observed
5	CPU Utilization	< 80%	~60%	Pass	Normal CPU usage during execution
6	Memory Utilization	< 80%	~65%	Pass	Memory usage remained stable

Step 4: Observations & Analysis

Key Findings:

The application successfully processed user requests and generated personalized networking suggestions. The response time was acceptable for a local AI-based application.

Bottlenecks Identified:

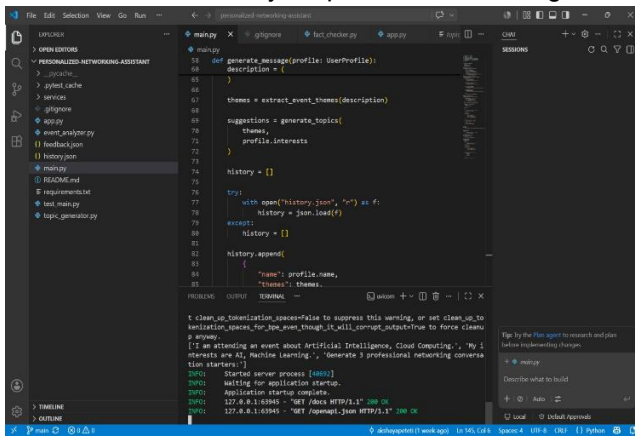
- AI model processing time affects response speed.
- External API response time depends on Wikipedia API availability.

Optimization Steps Taken:

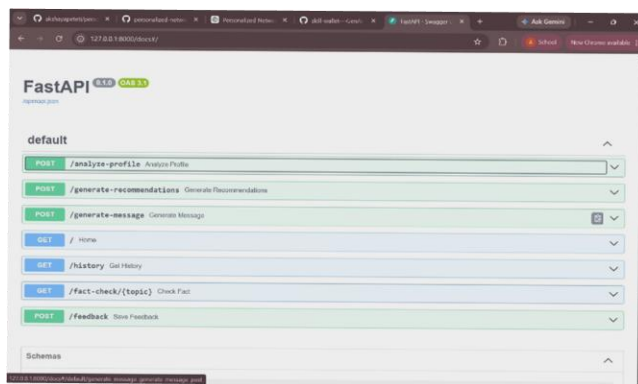
- Modular code structure for efficient processing.
- JSON-based lightweight storage.
- Separate frontend and backend architecture using Streamlit and FastAPI.

Step 5: Screenshots / Evidence

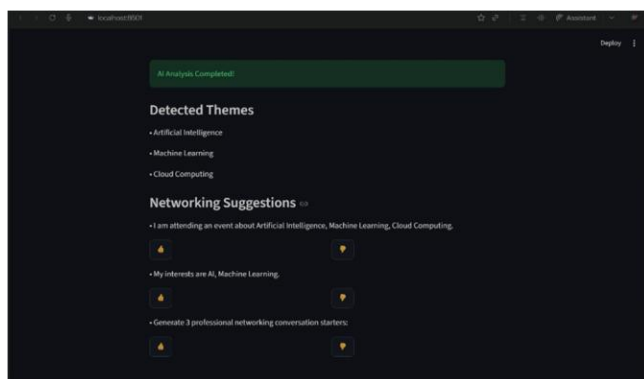
Attach screenshots of your performance testing tool results below :



1. Backend Server Running



2. API Documentation



3. Application Output